Data Acquisition System
Technical Report

Aquaphoton '27 Electrical Team

August 29, 2026

Abstract

This report documents the data acquisition firmware developed for the Aquaphoton '27 remotely operated car. The subsystem measures battery voltage, supplied current, three-axis acceleration, and three-axis angular velocity. An analog voltage-divider interface and an ACS712 current sensor are sampled using the microcontroller's analog-to-digital converter (ADC), while an MPU6050 inertial measurement unit (IMU) is read over the I^2C bus. The firmware converts raw measurements into engineering units, applies fixed IMU offsets, and transmits a human-readable telemetry frame over the serial port. This document explains the architecture, calculations, firmware sequence, interface format, calibration, and validation procedure for the implemented system.

Contents

1 Introduction

The data acquisition subsystem provides electrical and motion feedback from the vehicle to the control station. According to the project requirements, the firmware must acquire battery voltage, supplied current, and IMU measurements accurately and make them available for transmission to a Robot Operating System (ROS) node and, ultimately, the graphical user interface (GUI).

The current implementation is contained in `Dataacquisition.ino`. It performs local acquisition and serial reporting. The ROS parser, wireless link, warning logic, and GUI are outside the scope of this firmware file and therefore are treated as integration interfaces rather than completed functions in this report.

1.1 Objectives

The subsystem has the following objectives:

- measure the vehicle supply voltage through microcontroller pin PC1;
- measure the supplied current through an ACS712-5A sensor on microcontroller pin PC0;
- acquire acceleration and angular-rate data from an MPU6050 IMU;
- compensate the IMU readings using experimentally determined constant offsets;
- convert all raw data to volts, amperes, multiples of gravitational acceleration (g), and degrees per second (°/s); and
- send a complete telemetry line through a 9600-baud serial connection.

2 System Architecture

Figure 1 shows the implemented acquisition path and the intended station-side integration.

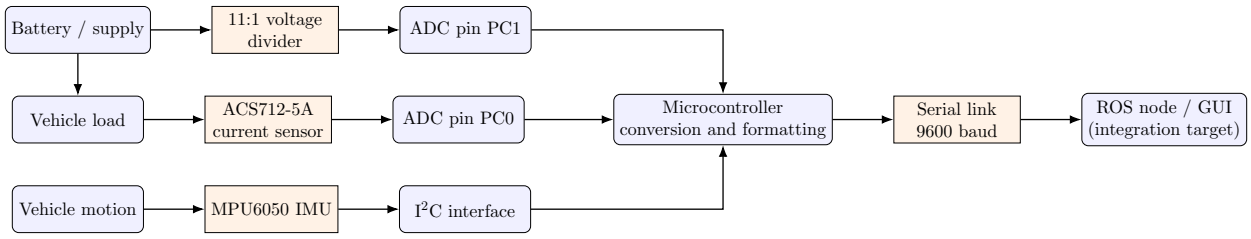

Figure 1: Data acquisition architecture. The ROS and GUI blocks show the required downstream interface, not code implemented in the supplied firmware.

Table 1: Acquired signals and firmware interfaces.

Quantity	Sensor/interface	MCU connection	Reported unit
Battery voltage	Resistive voltage divider	PC1	V
Supply current	ACS712-5A	PC0	A
Acceleration	MPU6050 accelerometer	I^2C, address `0x68`	g
Angular rate	MPU6050 gyroscope	I^2C, address `0x68`	$^\circ$/s
Telemetry	UART serial	Host serial interface	Labelled ASCII

2.1 Signal Summary

3 Hardware Interfaces and Calculations

3.1 Battery-Voltage Measurement

The voltage channel is connected to PC1. The firmware assumes a 10-bit ADC, a 5.0 V ADC reference, and a voltage-divider ratio of 11.0. If N_V is the ADC code, the ADC-pin voltage is

$$V_{\mathrm{ADC}} = \frac{N_V}{1023}(5.0\ \mathrm{V}), \tag{1}$$

and the estimated battery voltage is

$$V_{\mathrm{battery}} = 11 V_{\mathrm{ADC}} = \frac{N_V}{1023}(55.0\ \mathrm{V}). \tag{2}$$

The nominal input resolution is therefore

$$\Delta V_{\mathrm{battery}} = \frac{55.0}{1023} \approx 53.8\ \mathrm{mV/count}. \tag{3}$$

The theoretical full-scale value from the constants is 55 V. This is not automatically a safe operating voltage: the actual limit must also account for resistor voltage and power ratings, ADC protection, transients, PCB spacing, and the exact microcontroller supply and reference voltage. A fuse and suitable input protection are recommended for the final vehicle PCB.

For a divider made from an upper resistor R_1 and lower resistor R_2, the required relationship is

$$\frac{R_1 + R_2}{R_2} = 11, \tag{4}$$

for example, nominal values of $R_1 = 100\ \mathrm{k\Omega}$ and $R_2 = 10\ \mathrm{k\Omega}$. The installed resistor values must be measured and the ratio in the firmware updated to reduce gain error.

3.2 Current Measurement

The current channel is connected to PC0. The code is configured for the 5 A ACS712 variant, with a nominal zero-current output of 2.5 V and sensitivity of 0.185 V/A. If N_I is the ADC code, then

$$V_{\text{sensor}} = \frac{N_I}{1023}(5.0\text{ V}), \tag{5}$$

$$I = \frac{V_{\text{sensor}} - 2.5\text{ V}}{0.185\text{ V/A}}. \tag{6}$$

The nominal current resolution associated with one ADC count is

$$\Delta I = \frac{5.0/1023}{0.185} \approx 26.4\text{ mA/count}. \tag{7}$$

The ACS712 output is bidirectional: a value above its zero-current voltage produces a positive result, while a value below it produces a negative result. The actual zero point varies with supply voltage, device tolerance, temperature, ADC reference, and noise. Consequently, `ZERO_CURRENT_VOLTAGE` should be calibrated on the assembled system with no load current. The supplied ACS712 reference is listed in [1].

3.3 MPU6050 Interface

The MPU6050 is addressed at `0x68`; the supplied MPU6050 reference is listed in [2]. During setup, the firmware writes zero to power-management register `0x6B`, waking the device. Each acquisition starts at register `0x3B` and requests 14 consecutive bytes. These bytes contain three accelerometer axes, temperature, and three gyroscope axes. The temperature bytes are read and discarded.

No full-scale configuration registers are changed, so the code assumes the reset/default measurement ranges of $\pm 2g$ for acceleration and $\pm 250^\circ$/s for angular rate. Under these assumed settings, the conversion factors used by the program are

$$a_i = \frac{R_{a_i} - O_{a_i}}{16384} \quad [g], \tag{8}$$

$$\omega_i = \frac{R_{g_i} - O_{g_i}}{131} \quad [^\circ/\text{s}], \tag{9}$$

where R is a signed 16-bit raw reading and O is its stored offset. Table 2 lists the offsets currently hard-coded in the firmware.

Table 2: Implemented MPU6050 raw-count offsets.

Sensor	X axis	Y axis	Z axis
Accelerometer	148.66	−80.82	948.22
Gyroscope	283.06	214.34	132.27

The accelerometer reports specific force, not a gravity-free translational acceleration. Thus, when the vehicle is stationary, the axis aligned with gravity should measure approximately $\pm 1g$. The

constant-offset correction removes bias only; it does not correct scale-factor error, axis misalignment, vibration, or temperature drift. In addition, the present firmware reports angular rate rather than calculating absolute orientation. Estimating roll, pitch, or yaw requires sensor fusion or time integration with drift management.

4 Firmware Design

4.1 Initialization

The `setup()` function performs three operations:

1. starts UART communication at 9600 baud;
2. initializes the I^2C peripheral; and
3. wakes the MPU6050 by clearing its sleep bit through register `0x6B`.

It then sends `System Started` to indicate that initialization has completed.

4.2 Acquisition Cycle

The main loop first calls `readMPU6050()`, then samples voltage and current once each. It prints the converted results and waits 500 ms before beginning the next cycle. The explicit delay gives a nominal upper bound of two acquisition cycles per second; serial transmission and sensor access make the achieved rate slightly lower.

4.3 Telemetry Format

Each measurement cycle produces one line in the following format:

```
V: <voltage> V, I: <current> A, AX: <ax>, AY: <ay>, AZ: <az>,
GX: <gx>, GY: <gy>, GZ: <gz>
```

Voltage and current are printed with two digits after the decimal point; IMU values are printed with three. The labels make manual debugging easy but increase bandwidth and require a tolerant station-side parser. A ROS bridge can read one line at a time, validate that all eight fields are present, convert them to numeric values, timestamp the complete sample, and publish them to suitable topics.

Recommended ROS message choices are `sensor_msgs/Imu` for IMU data and either custom messages or `std_msgs/Float32` topics for voltage and current. The present accelerometer values are in g and gyroscope values are in degrees per second; a standards-compliant `sensor_msgs/Imu` publisher should convert these to m/s^2 and rad/s respectively.

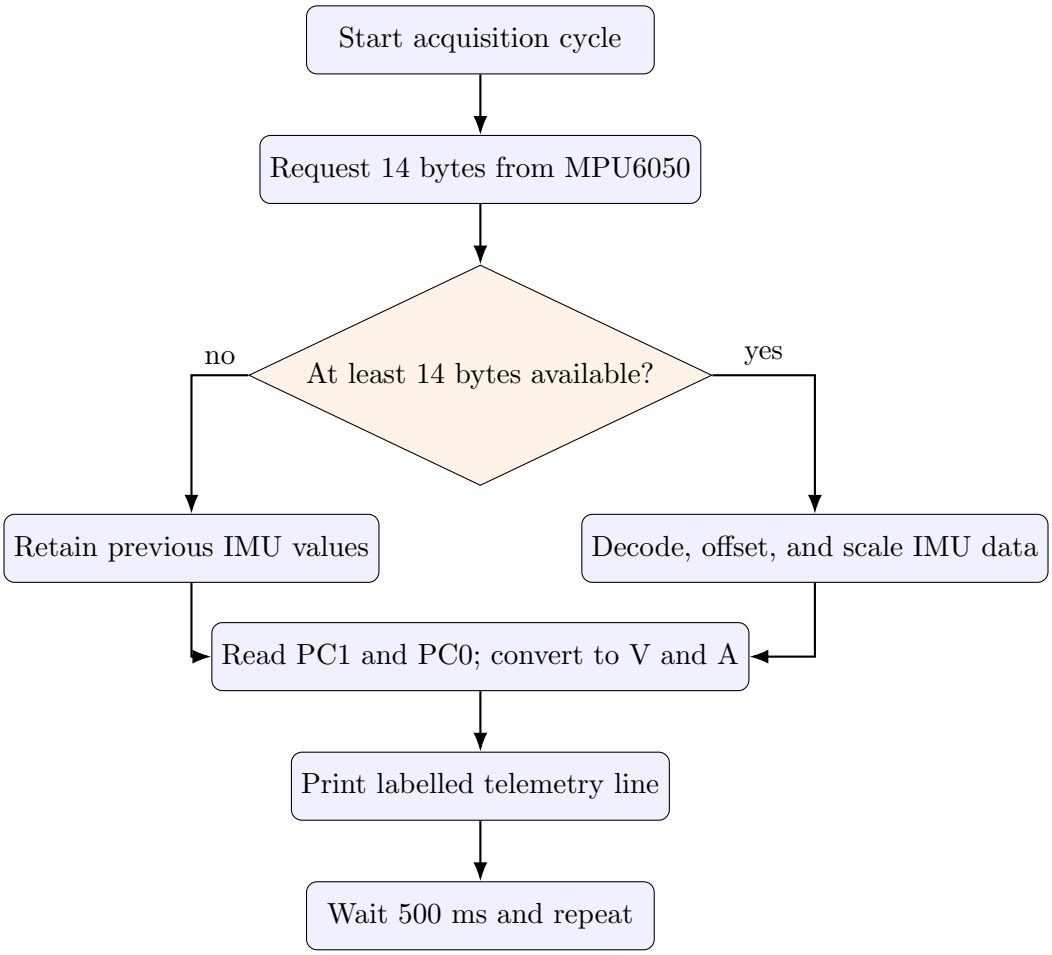

Figure 2: Flowchart of the implemented acquisition loop.

5 Calibration Procedure

5.1 Current Channel

The current-measurement stage uses the ACS712 Hall-effect sensor, whose analog output voltage varies linearly with the current flowing through its primary conduction path. According to the sensor datasheet, the output at zero current is nominally equal to half the supply voltage, $V_{\mathrm{CC}}/2$. Since the sensor is powered from a 5 V supply, the nominal zero-current offset is therefore

$$V_0 = \frac{V_{\mathrm{CC}}}{2} = 2.5 \text{ V}. \tag{10}$$

This offset provides the reference point for converting the measured output voltage into current. The general conversion relationship is

$$I = \frac{V_{\mathrm{measured}} - V_0}{S}, \tag{11}$$

where V_{measured} is the voltage measured at the ACS712 output, V_0 is the zero-current offset, and S is the sensor sensitivity. For the ACS712-05B 5 A variant, the nominal sensitivity is 185 mV/A, or 0.185 V/A. Substitution of the datasheet values gives

$$I = \frac{V_{\mathrm{measured}} - 2.5}{0.185} \quad [\mathrm{A}]. \tag{12}$$

The present firmware uses the manufacturer's nominal values of 2.5 V for the zero-current offset and 0.185 V/A for the sensitivity. These constants are used because the physical sensor has not yet been available for direct empirical calibration. Although the datasheet values provide an appropriate initial conversion, the actual offset and sensitivity of an individual device may differ slightly because of normal manufacturing tolerance, supply variation, temperature, and ADC error.

An on-device calibration routine has been prepared for use when the hardware becomes available. Under zero-current conditions, the routine will average multiple ADC samples to determine the actual offset voltage of the installed sensor. A known reference current, verified using a trusted multimeter, will then be applied to determine its effective sensitivity. The measured calibration constants can subsequently replace the nominal datasheet values in the firmware, reducing systematic measurement error and improving the accuracy of the current feedback.

5.2 IMU

Keep the IMU stationary on a level, vibration-free surface and average several hundred readings. Gyroscope means become zero-rate offsets. Accelerometer calibration should preserve the $1g$ gravity vector rather than forcing all three axes to zero. A six-position calibration—placing each axis in both the $+1g$ and $-1g$ directions—can estimate both offset and scale for every accelerometer axis.

6 Conclusion

The implemented subsystem acquires all three required classes of feedback: battery voltage, supplied current, and six-axis IMU data. Its calculations are consistent with a 5 V, 10-bit ADC, an 11:1 voltage divider, an ACS712-5A sensor, and an MPU6050 in its assumed default ranges. The output is suitable for bench inspection and provides a clear starting point for a ROS bridge. Calibration against reference instruments will establish the final measurement accuracy before integration with the vehicle GUI.

References

[1] AllDataSheet, *ACS712 Datasheet Search and Device Documentation*. Available at: `https://www.alldatasheet.com/view.jsp?Searchword=Acs712`. Accessed August 29, 2026.

[2] Ultra Librarian, *MPU6050 Datasheet Explained*. Available at: `http://www.ultralibrarian.com/2025/12/16/mpu6050-datasheet-explained-ulc/`. Accessed August 29, 2026.

A Supplied Firmware Listing

```
#include <Wire.h>

const byte MPU6050_ADDRESS = 0x68;

int16_t Rax, Ray, Raz;
int16_t Rgx, Rgy, Rgz;

float ax, ay, az;
float gx, gy, gz;

const float AX_OFFSET = 148.66;
const float AY_OFFSET = -80.82;
const float AZ_OFFSET = 948.22;

const float GX_OFFSET = 283.06;
const float GY_OFFSET = 214.34;
const float GZ_OFFSET = 132.27;

const int VOLTAGE_PIN = PC1 ;
const float VOLTAGE_DIVIDER_RATIO = 11 ;
const float ADC_REFERENCE = 5.0;

const int CURRENT_PIN = PC0 ;
const float ZERO_CURRENT_VOLTAGE = 2.5;
const float ACS712_SENSITIVITY = 0.185;

float readVoltage()
{
  int adcValue = analogRead(VOLTAGE_PIN);
  float adcVoltage = (adcValue * ADC_REFERENCE) / 1023.0;
  float batteryVoltage = adcVoltage * VOLTAGE_DIVIDER_RATIO;
  return batteryVoltage;
```

```
}

float readCurrent()
{
  int adcValue = analogRead(CURRENT_PIN);
  float sensorVoltage = (adcValue * ADC_REFERENCE) / 1023.0;
  float current = (sensorVoltage - ZERO_CURRENT_VOLTAGE) / ACS712_SENSITIVITY;
  return current;
}

void readMPU6050()
{
  Wire.beginTransmission(MPU6050_ADDRESS);
  Wire.write(0x3B);
  Wire.endTransmission(false);

  Wire.requestFrom(MPU6050_ADDRESS, 14);

  if (Wire.available() >= 14)
  {
    Rax = (Wire.read() << 8) | Wire.read();
    Ray = (Wire.read() << 8) | Wire.read();
    Raz = (Wire.read() << 8) | Wire.read();

    Wire.read();
    Wire.read();

    Rgx = (Wire.read() << 8) | Wire.read();
    Rgy = (Wire.read() << 8) | Wire.read();
    Rgz = (Wire.read() << 8) | Wire.read();

    float calibratedRax = Rax - AX_OFFSET;
    float calibratedRay = Ray - AY_OFFSET;
    float calibratedRaz = Raz - AZ_OFFSET;

    float calibratedRgx = Rgx - GX_OFFSET;
    float calibratedRgy = Rgy - GY_OFFSET;
    float calibratedRgz = Rgz - GZ_OFFSET;

    ax = calibratedRax / 16384.0;
    ay = calibratedRay / 16384.0;
    az = calibratedRaz / 16384.0;

    gx = calibratedRgx / 131.0;
    gy = calibratedRgy / 131.0;
    gz = calibratedRgz / 131.0;
  }
}

void setup()
{
  Serial.begin(9600);
  Wire.begin();

  Wire.beginTransmission(MPU6050_ADDRESS);
  Wire.write(0x6B);
  Wire.write(0x00);
  Wire.endTransmission();

```

```
  Serial.println("System Started");
}

void loop()
{
  readMPU6050();

  float voltage = readVoltage();
  float current = readCurrent();

  Serial.print("V: ");
  Serial.print(voltage, 2);

  Serial.print(" V, I: ");
  Serial.print(current, 2);

  Serial.print(" A, AX: ");
  Serial.print(ax, 3);

  Serial.print(", AY: ");
  Serial.print(ay, 3);

  Serial.print(", AZ: ");
  Serial.print(az, 3);

  Serial.print(", GX: ");
  Serial.print(gx, 3);

  Serial.print(", GY: ");
  Serial.print(gy, 3);

  Serial.print(", GZ: ");
  Serial.println(gz, 3);

  delay(500);
}
```